CS 341 — Algorithms

Vishesh Gupta

University of Waterloo

Contents

1 Introduction and Analysis Review **1**
- 1.1 Model of computation . . . 1
- 1.2 Asymptotic notation . . . 1
- 1.3 Analyzing loops . . . 2
- 1.4 The maximum subarray problem . . . 2
  - 1.4.1 Attempt 1: brute force, $\Theta(n^3)$ . . . 3
  - 1.4.2 Attempt 2: reuse partial sums, $\Theta(n^2)$ . . . 3
  - 1.4.3 Attempt 3: divide and conquer, $\Theta(n \log n)$ . . . 3
  - 1.4.4 Attempt 4: Kadane's algorithm, $\Theta(n)$ . . . 3
- 1.5 Proving correctness . . . 4

2 Recurrences and the Master Theorem **5**
- 2.1 Conventions . . . 5
- 2.2 Recursion trees . . . 5
- 2.3 The substitution method . . . 6
- 2.4 The Master theorem . . . 6
  - 2.4.1 Proof sketch of the Master theorem . . . 7
- 2.5 Other recurrence shapes . . . 7

3 Divide and Conquer **9**
- 3.1 Mergesort and counting inversions . . . 9
  - 3.1.1 Mergesort . . . 9
  - 3.1.2 Counting inversions . . . 9
- 3.2 Integer multiplication: Karatsuba . . . 10
- 3.3 Matrix multiplication: Strassen . . . 10
- 3.4 Closest pair of points . . . 11
- 3.5 Selection: median of medians . . . 11

4 Greedy Algorithms **13**
- 4.1 Interval scheduling . . . 13
- 4.2 Scheduling to minimize lateness . . . 14
- 4.3 Fractional knapsack . . . 14
- 4.4 Huffman codes . . . 15
- 4.5 Pitfalls . . . 15

5 Dynamic Programming **17**
5.1 Weighted interval scheduling . . . 17
5.2 0/1 knapsack . . . 18
5.3 Longest common subsequence (LCS) . . . 18
5.4 Edit distance . . . 19
5.5 Longest increasing subsequence (LIS) . . . 19
5.6 Matrix-chain multiplication . . . 19
5.7 Optimal binary search trees . . . 20
5.8 Top-down memoization . . . 20
5.9 Pitfalls . . . 20

6 Graph Basics, BFS and DFS **22**
6.1 Definitions and representations . . . 22
6.2 Breadth-first search . . . 22
6.2.1 Testing bipartiteness . . . 23
6.3 Depth-first search . . . 23
6.3.1 Edge classification (directed graphs) . . . 23
6.4 Topological sort . . . 24
6.5 Strongly connected components . . . 24
6.5.1 Kosaraju's algorithm . . . 24
6.5.2 Tarjan's algorithm (one pass) . . . 25

7 Minimum Spanning Trees **26**
7.1 The cut property . . . 26
7.2 Kruskal's algorithm . . . 26
7.2.1 Union-find (disjoint sets) . . . 27
7.3 Prim's algorithm . . . 27
7.4 Uniqueness and other facts . . . 28

8 Shortest Paths **29**
8.1 Common structure . . . 29
8.2 Shortest paths in a DAG . . . 29
8.3 Dijkstra's algorithm . . . 30
8.4 Bellman–Ford . . . 30
8.5 All-pairs shortest paths: Floyd–Warshall . . . 31

9 Polynomial-Time Reductions and NP **33**
9.1 Decision problems and P . . . 33
9.2 Polynomial-time reductions . . . 34
9.2.1 First examples . . . 34
9.3 NP . . . 35
9.4 NP-hard and NP-complete . . . 35

10 NP-Completeness **37**
10.1 Satisfiability and the Cook–Levin theorem . . . 37
10.2 SAT $\leq_P$ 3-SAT . . . 38
10.3 3-SAT $\leq_P$ Independent Set . . . 38
10.4 Hamiltonian cycle . . . 38

10.4.1 3-SAT $\leq_P$ Directed Hamiltonian Cycle 38
10.4.2 Directed $\leq_P$ undirected Hamiltonian cycle 39
10.4.3 Ham-Cycle $\leq_P$ TSP 39
10.5 3-SAT $\leq_P$ Subset Sum 39
10.6 3-SAT $\leq_P$ 3-Colouring 40
10.7 Choosing a source problem 40

11 Undecidability and Coping with Hardness **42**
11.1 Decidability 42
11.2 The halting problem 42
11.3 Reductions for undecidability 43
11.4 Coping with NP-hardness 43
11.4.1 Special cases 43
11.4.2 Exact exponential algorithms 43
11.4.3 Approximation algorithms 44
11.4.4 Heuristics 44

Chapter 1

Introduction and Analysis Review

CS 341 is about *designing* algorithms and *proving* things about them: that they are correct, and how much time (and occasionally space) they use. The course is organized around design paradigms (divide and conquer, greedy, dynamic programming, graph search) and then turns to the limits of efficient computation (NP-completeness, undecidability). The main reference is CLRS (*Introduction to Algorithms*, Cormen, Leiserson, Rivest, Stein), with Kleinberg–Tardos used for some greedy and reduction material.

1.1 Model of computation

To talk about running time we need a model. The default is the **word RAM**:

- memory is an array of words, each holding an integer of $\mathcal{O}(\log n)$ bits (enough to store an index into the input);
- arithmetic $(+, -, \times, \div)$, comparisons, and memory reads/writes on single words each cost $\mathcal{O}(1)$.

The running time $T(n)$ of an algorithm is the maximum number of such elementary operations over all inputs of *size* n (worst-case analysis).

Remark. Input *size* matters. For a list of n integers the size is usually taken as n (assuming each fits in a word). For a number-theoretic problem whose input is a single integer N, the size is the number of bits, $\log N$. An algorithm that runs in time $\mathcal{O}(N)$ on input N is then *exponential* in the input size. This distinction comes back for knapsack (Chapter 5) and subset sum (Chapter 10).

We also use the **bit complexity** model occasionally (e.g. for integer multiplication in Chapter 3), where each bit operation costs 1.

1.2 Asymptotic notation

Definition 1.1. Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$.

- $f(n) \in \mathcal{O}(g(n))$ if there exist $c > 0$ and n_0 such that $f(n) \leq c\, g(n)$ for all $n \geq n_0$.
- $f(n) \in \Omega(g(n))$ if there exist $c > 0$ and n_0 such that $f(n) \geq c\, g(n)$ for all $n \geq n_0$.
- $f(n) \in \Theta(g(n))$ if $f \in \mathcal{O}(g)$ and $f \in \Omega(g)$.

- $f(n) \in o(g(n))$ if for *every* $c > 0$ there is n_0 with $f(n) \le c\, g(n)$ for all $n \ge n_0$.
- $f(n) \in \omega(g(n))$ if for *every* $c > 0$ there is n_0 with $f(n) \ge c\, g(n)$ for all $n \ge n_0$.

A convenient tool is the **limit test**: if $L = \lim_{n\to\infty} f(n)/g(n)$ exists, then

$$L = 0 \Rightarrow f \in o(g), \qquad 0 < L < \infty \Rightarrow f \in \Theta(g), \qquad L = \infty \Rightarrow f \in \omega(g).$$

(The converse fails: f can be $\Theta(g)$ even if the limit does not exist, e.g. $f(n) = (2 + (-1)^n)n$ and $g(n) = n$.)

Example 1.2. $\log n \in o(n^\varepsilon)$ for every $\varepsilon > 0$: by L'Hôpital, $\lim \frac{\ln n}{n^\varepsilon} = \lim \frac{1/n}{\varepsilon n^{\varepsilon-1}} = \lim \frac{1}{\varepsilon n^\varepsilon} = 0$. Similarly $n^k \in o(c^n)$ for any constant k and $c > 1$.

Growth-rate ladder

$$1 \prec \log\log n \prec \log n \prec \sqrt{n} \prec n \prec n\log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n$$

Also: $\log(n!) \in \Theta(n \log n)$ (Stirling), $\sum_{i=1}^{n} 1/i = \ln n + \mathcal{O}(1)$, $\sum_{i=1}^{n} i^k \in \Theta(n^{k+1})$, and a geometric series $\sum_{i=0}^{n} r^i$ is $\Theta(1)$ if $r < 1$, $\Theta(n)$ if $r = 1$, $\Theta(r^n)$ if $r > 1$.

Useful properties: $\mathcal{O}$ is transitive; $f + g \in \Theta(\max(f, g))$; $\log_a n \in \Theta(\log_b n)$ for constant bases $a, b > 1$, so the base of a log is irrelevant inside $\mathcal{O}$.

1.3 Analyzing loops

For nested loops, write the cost as a sum and evaluate.

```
for i = 1 to n do
    for j = i to n do
        for k = i to j do
            Θ(1) work
```

The cost is $\sum_{i=1}^{n} \sum_{j=i}^{n} (j - i + 1) = \sum_{i=1}^{n} \binom{n-i+2}{2} \in \Theta(n^3)$. A quick lower bound argument: for $i \le n/4$ and $j \ge 3n/4$ the inner loop runs at least $n/2$ times, and there are $(n/4)^2$ such pairs, giving $\Omega(n^3)$.

> To prove a Θ bound for a loop, it is often easiest to prove an $\mathcal{O}$ upper bound by bounding every term by the largest, and an Ω lower bound by keeping only a constant fraction of the terms, each of which is large.

1.4 The maximum subarray problem

Definition 1.3 (Maximum subarray)**.** Given an array $A[1..n]$ of (possibly negative) integers, find indices $i \le j$ maximizing $A[i] + A[i+1] + \cdots + A[j]$. (We allow the empty subarray with sum 0, so the answer is never negative.)

For example, with $A = [2, -6, 3, -1, 4, -5, 2]$ the best is $[3, -1, 4]$ with sum 6. This problem is the running example for seeing how algorithm design can take us from $\Theta(n^3)$ to $\Theta(n)$.

1.4.1 Attempt 1: brute force, $\Theta(n^3)$

```
best = 0
for i = 1 to n do
    for j = i to n do
        s = 0
        for k = i to j do s = s + A[k]
        best = max(best, s)
return best
```

This is exactly the triple loop analyzed above: $\Theta(n^3)$.

1.4.2 Attempt 2: reuse partial sums, $\Theta(n^2)$

The sum $A[i..j]$ equals $A[i..j-1] + A[j]$, so we can maintain it incrementally.

```
best = 0
for i = 1 to n do
    s = 0
    for j = i to n do
        s = s + A[j]          // s = A[i] + ... + A[j]
        best = max(best, s)
return best
```

Equivalently, precompute prefix sums $P[j] = A[1] + \cdots + A[j]$ (with $P[0] = 0$); then the sum of $A[i..j]$ is $P[j] - P[i-1]$.

1.4.3 Attempt 3: divide and conquer, $\Theta(n \log n)$

Split A at the middle $m = \lfloor (1+n)/2 \rfloor$. An optimal subarray lies entirely in the left half, entirely in the right half, or crosses the midpoint. The first two cases are solved recursively. For the crossing case, the best crossing subarray is (best suffix of $A[1..m]$) + (best prefix of $A[m+1..n]$), each found by a linear scan.

```
MaxSub(A, lo, hi):
    if lo == hi then return max(0, A[lo])
    m = floor((lo + hi) / 2)
    L = MaxSub(A, lo, m);  R = MaxSub(A, m+1, hi)
    s = 0; bestL = 0
    for i = m downto lo do  s = s + A[i]; bestL = max(bestL, s)
    s = 0; bestR = 0
    for j = m+1 to hi do    s = s + A[j]; bestR = max(bestR, s)
    return max(L, R, bestL + bestR)
```

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$, so $T(n) \in \Theta(n \log n)$ (Chapter 2).

1.4.4 Attempt 4: Kadane's algorithm, $\Theta(n)$

Let $E[j]$ be the maximum sum of a subarray *ending exactly at* j (possibly empty). Either the best such subarray is empty, or it is $A[j]$ appended to the best subarray ending at $j-1$:

$$E[0] = 0, \qquad E[j] = \max(0,\ E[j-1] + A[j]).$$

The answer is $\max_j E[j]$.

```
best = 0; cur = 0
for j = 1 to n do
    cur = max(0, cur + A[j])
    best = max(best, cur)
return best
```

Theorem 1.4. *Kadane's algorithm returns the maximum subarray sum and runs in $\Theta(n)$ time.*

Proof. Runtime is clear: one pass with $\mathcal{O}(1)$ work per element. For correctness, we prove by induction on j the loop invariant: after iteration j, `cur` $= E[j]$ and `best` $= \max_{0 \le t \le j} E[t]$. Base $j = 0$: both are $0 = E[0]$. Step: any subarray ending at j is either empty (sum 0) or $A[i..j]$ for some $i \le j$; in the latter case its sum is $A[i..j-1] + A[j] \le E[j-1] + A[j]$, and the value $E[j-1] + A[j]$ is achieved by extending an optimal subarray ending at $j-1$. Hence $E[j] = \max(0, E[j-1] + A[j])$, which is what `cur` becomes. Every subarray ends at some j, so the maximum over all subarrays is $\max_j E[j]$, which is `best` at the end. □

Any algorithm must read every element (an unread element could be huge), so $\Omega(n)$ is a lower bound and Kadane is optimal.

Lesson

The four solutions illustrate the main themes of the course: a naive algorithm, then removing redundant work, then divide and conquer, then a dynamic programming recurrence. The last one is optimal, and proving that needed a loop invariant.

1.5 Proving correctness

Typical tools:

- **Loop invariants**: a statement true before the first iteration (initialization), preserved by each iteration (maintenance), and which at termination gives correctness.
- **Induction on input size** for recursive algorithms: assume the recursive calls are correct, and show the combine step is correct.
- **Exchange arguments** for greedy algorithms (Chapter 4).
- **Optimal substructure** for dynamic programming (Chapter 5).

Summary

- Running time is measured in the word RAM model, worst case, as a function of input size.
- Know the definitions of $\mathcal{O}, \Omega, \Theta, o, \omega$, the limit test, and the growth ladder.
- Maximum subarray: $\Theta(n^3)$ brute force $\to \Theta(n^2)$ incremental $\to \Theta(n \log n)$ divide and conquer $\to \Theta(n)$ Kadane.
- Correctness is proved with invariants or induction; input size is the number of bits for numeric inputs.

Chapter 2

Recurrences and the Master Theorem

Recursive algorithms have running times described by recurrences, such as $T(n) = 2T(n/2) + \Theta(n)$ for mergesort. This chapter covers three ways to solve them: recursion trees (to guess), substitution (to prove), and the Master theorem (to look up).

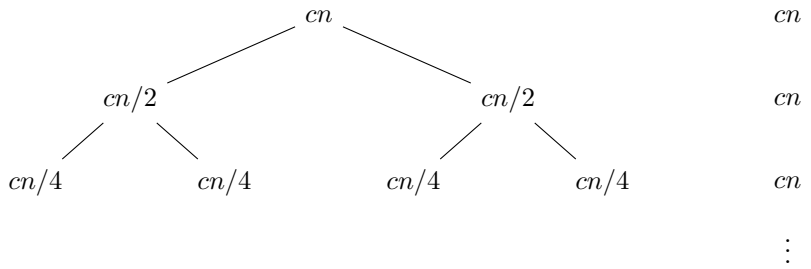
2.1 Conventions

- We usually ignore floors and ceilings: $T(\lfloor n/2 \rfloor)$ and $T(\lceil n/2 \rceil)$ are written $T(n/2)$. This does not change the asymptotic answer for the recurrences in this course (it can be justified rigorously, e.g. by proving the bound for powers of 2 and using monotonicity).
- The base case $T(n) = \Theta(1)$ for $n \leq n_0$ is implicit.
- "$+f(n)$" means the non-recursive work: dividing and combining.

2.2 Recursion trees

Draw the recursion as a tree: the root costs $f(n)$, it has a children each of size n/b, each costing $f(n/b)$, and so on. Sum the costs level by level.

Example 2.1 ($T(n) = 2T(n/2) + cn$)**.** Level i has 2^i nodes each of size $n/2^i$, costing $cn/2^i$ each, so cn per level. There are $\log_2 n + 1$ levels, so $T(n) = cn(\log_2 n + 1) \in \Theta(n \log n)$.



Example 2.2 ($T(n) = 3T(n/4) + cn^2$)**.** Level i: 3^i nodes, each costing $c(n/4^i)^2$, total $cn^2(3/16)^i$. This is a decreasing geometric series, so $T(n) \leq cn^2 \sum_{i \geq 0} (3/16)^i = \frac{16}{13} cn^2$, and $T(n) \in \Theta(n^2)$: the root dominates.

Example 2.3 ($T(n) = 4T(n/2) + cn$)**.** Level i costs $4^i \cdot cn/2^i = cn2^i$, an increasing geometric series dominated by the leaves. There are $4^{\log_2 n} = n^2$ leaves, so $T(n) \in \Theta(n^2)$.

Example 2.4 (Unbalanced: $T(n) = T(n/3)+T(2n/3)+cn$)**.** Every full level costs cn. The shortest root-to-leaf path has length $\log_3 n$ and the longest $\log_{3/2} n$. So $cn \log_3 n \le T(n) \le cn \log_{3/2} n$ roughly, i.e. $T(n) \in \Theta(n \log n)$.

2.3 The substitution method

Guess the answer (often from a recursion tree), then prove it by induction, with explicit constants.

Example 2.5. Claim: $T(n) = 2T(\lfloor n/2 \rfloor) + n$ with $T(1) = 1$ satisfies $T(n) \le cn \log_2 n$ for $n \ge 2$ and suitable c.

Inductive step: assuming the bound for $\lfloor n/2 \rfloor$,

$$T(n) \le 2c \lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor + n \le cn \log_2(n/2) + n = cn \log_2 n - cn + n \le cn \log_2 n$$

whenever $c \ge 1$. For the base cases, $T(2) = 4$ and $T(3) = 5$, so $c \ge 2$ works ($2 \cdot 2 \log 2 = 4$, $2 \cdot 3 \log_2 3 \approx 9.5$). All $n \ge 4$ reduce to $\lfloor n/2 \rfloor \ge 2$, so the induction is sound.

Pitfall: proving the exact form

In the inductive step you must derive *exactly* the hypothesis $T(n) \le c\,g(n)$, not $T(n) \le c\,g(n)+n$. "$T(n) \le cn+n \in \mathcal{O}(n)$" is **not** a valid proof that $T(n) \le cn$; the constant cannot grow at each step.

Strengthening the hypothesis. Sometimes the natural guess fails by a lower-order term and we must subtract one. For $T(n) = 2T(n/2) + 1$, guessing $T(n) \le cn$ gives $T(n) \le cn + 1$, which does not close. Instead guess $T(n) \le cn - d$: then $T(n) \le 2(cn/2 - d) + 1 = cn - 2d + 1 \le cn - d$ when $d \ge 1$. So $T(n) \in \mathcal{O}(n)$.

Changing variables. For $T(n) = 2T(\sqrt{n}) + \log n$, set $m = \log n$ and $S(m) = T(2^m)$. Then $S(m) = 2S(m/2) + m$, so $S(m) = \Theta(m \log m)$ and $T(n) = \Theta(\log n \log\log n)$.

2.4 The Master theorem

Theorem 2.6 (Master theorem, CLRS form)**.** *Let $a \ge 1$, $b > 1$, and $T(n) = aT(n/b) + f(n)$. Let $c^* = \log_b a$ (the* critical exponent*).*

1. *If $f(n) \in \mathcal{O}(n^{c^*-\varepsilon})$ for some $\varepsilon > 0$, then $T(n) \in \Theta(n^{c^*})$.* *(leaves dominate)*
2. *If $f(n) \in \Theta(n^{c^*} \log^k n)$ for some $k \ge 0$, then $T(n) \in \Theta(n^{c^*} \log^{k+1} n)$.* *(balanced)*
3. *If $f(n) \in \Omega(n^{c^*+\varepsilon})$ for some $\varepsilon > 0$, and the* regularity condition *$af(n/b) \le \delta f(n)$ holds for some $\delta < 1$ and large n, then $T(n) \in \Theta(f(n))$.* *(root dominates)*

The intuition is the recursion tree: there are $a^{\log_b n} = n^{\log_b a}$ leaves, and level i costs $a^i f(n/b^i)$. The three cases say the level costs are increasing geometrically, roughly constant, or decreasing geometrically.

Simplified version for $f(n) = \Theta(n^y)$. Compare y with $x = \log_b a$:

$$T(n) \in \begin{cases} \Theta(n^x) & y < x, \\ \Theta(n^x \log n) & y = x, \\ \Theta(n^y) & y > x. \end{cases}$$

(The regularity condition holds automatically for polynomials.)

Example 2.7 (Standard recurrences)**.**

Recurrence	$\log_b a$	Case	Solution
$T(n) = 2T(n/2) + n$ (mergesort)	1	2	$\Theta(n \log n)$
$T(n) = T(n/2) + 1$ (binary search)	0	2	$\Theta(\log n)$
$T(n) = 3T(n/2) + n$ (Karatsuba)	$\log_2 3 \approx 1.585$	1	$\Theta(n^{\log_2 3})$
$T(n) = 7T(n/2) + n^2$ (Strassen)	$\log_2 7 \approx 2.807$	1	$\Theta(n^{\log_2 7})$
$T(n) = 8T(n/2) + n^2$ (naive block matmult)	3	1	$\Theta(n^3)$
$T(n) = 2T(n/2) + n \log n$	1	2 ($k = 1$)	$\Theta(n \log^2 n)$
$T(n) = T(n/2) + n$	0	3	$\Theta(n)$

When the Master theorem does not apply

- Subproblems of different sizes, e.g. $T(n) = T(n/5) + T(7n/10) + n$ (median of medians); use a recursion tree or substitution.
- Subtractive recurrences, e.g. $T(n) = T(n-1) + n$, which gives $\Theta(n^2)$ by summing.
- Gaps between cases: $T(n) = 2T(n/2) + n/\log n$ is not covered by the CLRS form (the answer is $\Theta(n \log \log n)$).

2.4.1 Proof sketch of the Master theorem

Assume $n = b^k$. Unrolling,

$$T(n) = n^{\log_b a} T(1) + \sum_{i=0}^{k-1} a^i f(n/b^i).$$

For $f(n) = n^y$, the sum is $n^y \sum_{i=0}^{k-1} (a/b^y)^i$. Let $r = a/b^y$; note $r < 1 \iff y > \log_b a$.

- $r < 1$: the geometric sum is $\Theta(1)$, so $T(n) = \Theta(n^y)$ (the $n^{\log_b a}$ term is smaller).
- $r = 1$: the sum is $k = \log_b n$ terms of n^y, so $T(n) = \Theta(n^y \log n)$.
- $r > 1$: the sum is $\Theta(n^y r^k) = \Theta(n^y a^k / b^{yk}) = \Theta(a^k) = \Theta(n^{\log_b a})$.

2.5 Other recurrence shapes

- $T(n) = T(n-1) + \Theta(1) \Rightarrow \Theta(n)$; $T(n) = T(n-1) + \Theta(n) \Rightarrow \Theta(n^2)$.
- $T(n) = 2T(n-1) + \Theta(1) \Rightarrow \Theta(2^n)$ (e.g. naive recursion for Towers of Hanoi).
- $T(n) = T(n-1) + T(n-2) + \Theta(1) \Rightarrow \Theta(\varphi^n)$, $\varphi = (1 + \sqrt{5})/2$ (naive Fibonacci; motivates DP).

- $T(n) = T(\alpha n) + T(\beta n) + \Theta(n)$ with $\alpha + \beta < 1 \Rightarrow \Theta(n)$; with $\alpha + \beta = 1 \Rightarrow \Theta(n \log n)$.

Theorem 2.8. *If $T(n) \leq T(\alpha n) + T(\beta n) + cn$ with $\alpha + \beta < 1$, then $T(n) \in \mathcal{O}(n)$.*

Proof. Guess $T(n) \leq dn$. Then $T(n) \leq d\alpha n + d\beta n + cn = (d(\alpha + \beta) + c)n \leq dn$ provided $d \geq c/(1 - \alpha - \beta)$. Choose d also large enough for the base cases. □

Summary

- Recursion trees give a guess by summing per-level costs; decide whether the root, the leaves, or all levels dominate.
- Substitution proves the guess by induction; derive the exact hypothesis, and subtract lower-order terms if needed.
- Master theorem: compare $f(n)$ with $n^{\log_b a}$. Smaller $\Rightarrow n^{\log_b a}$; equal $\Rightarrow$ extra log; larger (with regularity) $\Rightarrow f(n)$.
- Uneven splits with total fraction < 1 cost $\Theta(n)$ when the extra work is linear.

Chapter 3

Divide and Conquer

Paradigm

Divide the instance into smaller instances of the same problem, **conquer** them recursively, and **combine** the sub-solutions. Correctness is by (strong) induction on the size; runtime is from a recurrence.

3.1 Mergesort and counting inversions

3.1.1 Mergesort

```
MergeSort(A[1..n]):
    if n <= 1 then return A
    L = MergeSort(A[1..n/2]);  R = MergeSort(A[n/2+1..n])
    return Merge(L, R)

Merge(L, R):                // L, R sorted
    i = 1; j = 1; out = []
    while i <= |L| and j <= |R| do
        if L[i] <= R[j] then append L[i] to out; i = i + 1
        else                 append R[j] to out; j = j + 1
    append the rest of L and R to out
    return out
```

`Merge` takes $\Theta(|L|+|R|)$, so $T(n) = 2T(n/2) + \Theta(n) \in \Theta(n \log n)$. Correctness of `Merge`: invariant "`out` holds the $i+j-2$ smallest elements of $L \cup R$, in sorted order".

3.1.2 Counting inversions

Definition 3.1. An *inversion* in $A[1..n]$ is a pair (i, j) with $i < j$ and $A[i] > A[j]$.

The number of inversions measures how far A is from sorted (between 0 and $\binom{n}{2}$); it is used, e.g., to compare two rankings. Brute force is $\Theta(n^2)$.

Divide and conquer: inversions are of three types: both indices in the left half, both in the right half, or *split* (i in the left, j in the right). The first two are counted recursively. To count split inversions quickly, sort both halves (which doesn't change the split count) and count during the merge: when $R[j]$ is output before the remaining elements $L[i..|L|]$, it forms an inversion with each of them, i.e. with $|L| - i + 1$ elements.

```
SortCount(A):                          // returns (sorted A, #inversions)
    if |A| <= 1 then return (A, 0)
    (L, a) = SortCount(left half);  (R, b) = SortCount(right half)
    i = 1; j = 1; c = 0; out = []
    while i <= |L| and j <= |R| do
        if L[i] <= R[j] then append L[i]; i = i + 1
        else append R[j]; c = c + (|L| - i + 1); j = j + 1
    append leftovers
    return (out, a + b + c)
```

Same recurrence as mergesort: $\Theta(n \log n)$.

3.2 Integer multiplication: Karatsuba

Multiplying two n-bit integers by the grade-school method takes $\Theta(n^2)$ bit operations. Split $x = x_1 2^{n/2} + x_0$ and $y = y_1 2^{n/2} + y_0$. Then

$$xy = x_1 y_1 2^n + (x_1 y_0 + x_0 y_1) 2^{n/2} + x_0 y_0.$$

Four recursive multiplications of $n/2$-bit numbers plus $\Theta(n)$ for shifts/additions gives $T(n) = 4T(n/2) + \Theta(n) \in \Theta(n^2)$: no gain.

Karatsuba's trick (1960): the middle term is

$$x_1 y_0 + x_0 y_1 = (x_1 + x_0)(y_1 + y_0) - x_1 y_1 - x_0 y_0,$$

so three multiplications suffice: $P_1 = x_1 y_1$, $P_2 = x_0 y_0$, $P_3 = (x_1 + x_0)(y_1 + y_0)$, and $xy = P_1 2^n + (P_3 - P_1 - P_2) 2^{n/2} + P_2$.

```
Karatsuba(x, y, n):
    if n <= threshold then return x * y        // grade school
    split x = (x1, x0), y = (y1, y0) at n/2 bits
    P1 = Karatsuba(x1, y1, n/2)
    P2 = Karatsuba(x0, y0, n/2)
    P3 = Karatsuba(x1 + x0, y1 + y0, n/2 + 1)
    return P1 * 2^n + (P3 - P1 - P2) * 2^(n/2) + P2
```

$T(n) = 3T(n/2) + \Theta(n)$, so $T(n) \in \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ by Master case 1. (The extra bit in $x_1 + x_0$ does not affect the asymptotics.) Toom–Cook and FFT-based methods do better (down to $\mathcal{O}(n \log n)$, Harvey–van der Hoeven 2019).

3.3 Matrix multiplication: Strassen

Naive multiplication of $n \times n$ matrices is $\Theta(n^3)$. Splitting into $n/2 \times n/2$ blocks,

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} E & F \\ G & H \end{pmatrix} = \begin{pmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{pmatrix}$$

uses 8 block products: $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$. **Strassen** (1969) uses 7 products:

$$M_1 = (A + D)(E + H), \quad M_2 = (C + D)E, \quad M_3 = A(F - H), \quad M_4 = D(G - E),$$
$$M_5 = (A + B)H, \quad M_6 = (C - A)(E + F), \quad M_7 = (B - D)(G + H),$$

and then

$$AE + BG = M_1 + M_4 - M_5 + M_7, \qquad AF + BH = M_3 + M_5,$$
$$CE + DG = M_2 + M_4, \qquad CF + DH = M_1 - M_2 + M_3 + M_6.$$

(You are not expected to memorize these; check one entry, e.g. $M_3 + M_5 = AF - AH + AH + BH$.) $T(n) = 7T(n/2) + \Theta(n^2) \in \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$. The point: *reducing the number of recursive calls* changes the exponent; extra additions only affect the $f(n)$ term.

3.4 Closest pair of points

Definition 3.2. Given n points in the plane, find a pair with minimum Euclidean distance.

Brute force is $\Theta(n^2)$. In 1D, sort and check neighbours: $\Theta(n \log n)$.

Algorithm. Presort the points by x (array P_x) and by y (array P_y).

1. Split by the median x-coordinate into Q (left) and R (right), each of size $n/2$. Also split P_y into Q_y, R_y in linear time (preserving y-order).
2. Recursively find δ_Q, δ_R; let $\delta = \min(\delta_Q, \delta_R)$.
3. Any closer pair must be split, with both points within δ of the dividing line $x = x^*$. Let S_y be the points of P_y with $|x - x^*| < \delta$, in y-order.
4. For each point in S_y, compare it with the next 7 points in S_y. Return the best found.

Lemma 3.3. *If $p, q \in S_y$ and $d(p, q) < \delta$, then p and q are within 7 positions of each other in S_y.*

Proof. Consider the $2\delta \times \delta$ rectangle above p (from p's y up to $y + \delta$, spanning the strip). Divide it into 8 squares of side $\delta/2$, four on each side of the line. Each square lies entirely on one side, and any two points in a square are at distance at most $\delta/\sqrt{2} < \delta$, contradicting the definition of δ (they would be on the same side). So each square holds at most one point, and the rectangle holds at most 8 points including p. Hence any q above p with $d(p, q) < \delta$ is among the next 7 entries. □

Runtime: presorting $\mathcal{O}(n \log n)$ once; then $T(n) = 2T(n/2) + \Theta(n) \in \Theta(n \log n)$. (Re-sorting by y at each level would give $\Theta(n \log^2 n)$.)

3.5 Selection: median of medians

Selection problem: given an unsorted array A of n distinct numbers and k, return the k-th smallest. Sorting gives $\Theta(n \log n)$; we want $\Theta(n)$.

Quickselect: pick a pivot p, partition into $L = \{x < p\}$, $G = \{x > p\}$; if $|L| = k - 1$ return p, if $k \le |L|$ recurse on L, else recurse on G with $k - |L| - 1$. With a random pivot the expected time is $\Theta(n)$, but the worst case is $\Theta(n^2)$.

Median of medians (Blum, Floyd, Pratt, Rivest, Tarjan 1973) chooses a pivot guaranteed to be good:

```
Select(A, k):
    if |A| <= 5 then sort A and return A[k]
    divide A into ⌈n/5⌉ groups of 5; find each group's median   // Θ(n)
    M = list of these medians
    p = Select(M, ⌈|M|/2⌉)            // median of medians
    partition A around p into L, {p}, G
    if k == |L| + 1 then return p
    elif k <= |L| then return Select(L, k)
    else return Select(G, k - |L| - 1)
```

Lemma 3.4. $|L| \le \frac{7n}{10} + \mathcal{O}(1)$ *and* $|G| \le \frac{7n}{10} + \mathcal{O}(1)$.

Proof. About half of the $n/5$ groups, i.e. at least $\left\lceil \frac{1}{2} \lceil n/5 \rceil \right\rceil$ groups, have median $\ge p$. Each such group (except possibly p's own group and a final partial group) contributes 3 elements $\ge p$ (its median and the two larger ones). So at least $3(\frac{n}{10} - 2) = \frac{3n}{10} - 6$ elements are $\ge p$, hence $|L| \le \frac{7n}{10} + 6$. Symmetrically for G. □

So $T(n) \le T(n/5) + T(7n/10 + 6) + \Theta(n)$. Since $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, the theorem from Chapter 2 gives $T(n) \in \Theta(n)$. (With groups of 3 the fractions sum to 1 and we only get $\Theta(n \log n)$; groups of 7 also work.)

Summary

- Inversions: count split inversions during the merge; $\Theta(n \log n)$.
- Karatsuba: 3 half-size products, $\Theta(n^{\log_2 3})$. Strassen: 7 products, $\Theta(n^{\log_2 7})$.
- Closest pair: strip argument, each point checks ≤ 7 neighbours in y-order; $\Theta(n \log n)$.
- Median of medians: pivot is guaranteed to discard $\approx 3n/10$ elements, giving $T(n) = T(n/5) + T(7n/10) + \Theta(n) = \Theta(n)$.

Chapter 4

Greedy Algorithms

Paradigm

A greedy algorithm builds a solution one piece at a time, each time making the choice that looks best *right now*, and never revisiting it. Greedy algorithms are easy to design and usually fast, but often wrong; the work is in the **proof**.

Two standard proof templates:

- **Greedy stays ahead**: show by induction that after each step, the greedy partial solution is at least as good as any other solution's corresponding prefix.
- **Exchange argument**: take an optimal solution O that differs from the greedy solution G; modify O step by step (swap two elements) into G without making it worse. Hence G is optimal.

4.1 Interval scheduling

Definition 4.1. Input: n intervals (jobs) $[s_i, f_i)$. Output: a maximum-size set of pairwise disjoint (compatible) intervals.

Candidate greedy rules: earliest start time (fails: one long early interval), shortest interval (fails: a short interval overlapping two long disjoint ones), fewest conflicts (fails on a more intricate example). The correct rule is **earliest finish time**.

```
sort intervals so that f_1 ≤ f_2 ≤ ··· ≤ f_n
S = {}; last = −∞
for i = 1 to n do
    if s_i ≥ last then
        S = S ∪ {i}; last = f_i
return S
```

Runtime $\Theta(n \log n)$ (sorting dominates).

Theorem 4.2. *Earliest-finish-first returns an optimal schedule.*

Proof (greedy stays ahead). Let $G = (g_1, \ldots, g_k)$ be the greedy intervals in order, and let $O = (o_1, \ldots, o_m)$ be an optimal solution sorted by finish time. Claim: $f(g_r) \le f(o_r)$ for all $r \le k$. For $r = 1$, greedy picks the interval with the smallest finish time overall. If $f(g_{r-1}) \le f(o_{r-1})$, then o_r

starts after $f(o_{r-1}) \geq f(g_{r-1})$, so o_r was compatible with greedy's choices when greedy chose g_r; greedy chose the earliest-finishing compatible interval, so $f(g_r) \leq f(o_r)$.

Now suppose $m > k$. Then o_{k+1} starts after $f(o_k) \geq f(g_k)$, so it is compatible with all of G. Greedy only stops when no compatible interval remains, so it would have selected another interval after g_k, a contradiction. Hence $k = m$. □

Interval colouring (partitioning). To schedule *all* intervals using the fewest rooms, process intervals by *start* time and put each in any free room (opening a new one if none). The number of rooms used equals the maximum depth (number of intervals over a common point), which is an obvious lower bound. With a min-heap of room finish times this is $\mathcal{O}(n \log n)$.

4.2 Scheduling to minimize lateness

Definition 4.3. Single machine; job i has processing time t_i and deadline d_i. A schedule orders the jobs with no idle time; job i finishes at f_i and has lateness $\ell_i = \max(0, f_i - d_i)$. Minimize the maximum lateness $L = \max_i \ell_i$.

Rule: **earliest deadline first** (EDF), ignoring processing times! (Shortest job first and smallest slack $d_i - t_i$ both fail.)

Theorem 4.4. *EDF minimizes maximum lateness.*

Proof (exchange argument). An *inversion* in a schedule is a pair of jobs i, j with $d_i < d_j$ but j scheduled before i. EDF has no inversions (ties in deadline can be ordered arbitrarily; all such orders have the same max lateness).

If a schedule has an inversion, it has an *adjacent* inversion: consecutive jobs j, i with $d_i < d_j$. Swap them. Other jobs are unaffected; i now finishes earlier, so ℓ_i does not increase. Job j now finishes at the time i used to finish, f, so its new lateness is $\max(0, f - d_j) \leq \max(0, f - d_i) =$ old ℓ_i. So the max lateness does not increase, and the number of inversions drops by one.

Start from an optimal schedule with no idle time; repeatedly swapping adjacent inversions gives an inversion-free schedule (i.e. EDF up to ties) that is still optimal. □

4.3 Fractional knapsack

Items $1..n$ with weights $w_i > 0$ and values v_i; capacity W. We may take a fraction $x_i \in [0, 1]$ of each item. Maximize $\sum x_i v_i$ subject to $\sum x_i w_i \leq W$.

Greedy: sort by value density v_i/w_i decreasing; take whole items while they fit, then a fraction of the next one. $\Theta(n \log n)$ (or $\Theta(n)$ using weighted-median selection).

Theorem 4.5. *The greedy algorithm is optimal for fractional knapsack.*

Proof. Order items so $v_1/w_1 \geq \cdots \geq v_n/w_n$; greedy's solution x is $(1, \ldots, 1, x_k, 0, \ldots, 0)$. (If all items fit, greedy takes everything and is trivially optimal, so assume $\sum w_i > W$; then greedy fills the knapsack exactly.) Let y be an optimal solution agreeing with x on as many leading indices as possible, and let i be the first index with $y_i \neq x_i$. Greedy takes as much of item i as fits, so $y_i < x_i$. Since $\sum_{j<i} y_j w_j = \sum_{j<i} x_j w_j$ and x fills the knapsack, either y has spare capacity or some item $j > i$ has $y_j > 0$. Move weight $\epsilon > 0$ (small) from such an item j, or from the spare capacity, to item i. The value changes by $\epsilon(v_i/w_i - v_j/w_j) \geq 0$. The new solution is feasible and still optimal; taking ϵ as large as possible either makes $y_i = x_i$ or empties item j, and repeating finitely often turns y into x without decreasing value. So x is optimal. □

Remark. Greedy by density **fails** for 0/1 knapsack. Take capacity $W = 50$ and three items with (weight, value) equal to $(10, 60)$, $(20, 100)$ and $(30, 120)$. Greedy takes items 1, 2 for value 160; optimal is items 2, 3 for 220. 0/1 knapsack needs dynamic programming (Chapter 5).

4.4 Huffman codes

We want a binary *prefix-free* code for an alphabet Σ (no codeword is a prefix of another), so a bit string decodes uniquely. A prefix code is a binary tree with characters at the leaves; the codeword is the root-to-leaf path (0 = left, 1 = right). Given frequencies $f(c)$, minimize the encoded length

$$B(T) = \sum_{c \in \Sigma} f(c) \cdot \mathrm{depth}_T(c).$$

```
Huffman(C, f):
    Q = min-priority queue of the characters keyed by f
    repeat |C| - 1 times:
        x = ExtractMin(Q); y = ExtractMin(Q)
        create node z with children x, y and f(z) = f(x) + f(y)
        Insert(Q, z)
    return ExtractMin(Q)            // root of the tree
```

With a binary heap, $\mathcal{O}(n \log n)$ for $n = |\Sigma|$.

Example 4.6. Frequencies a:45, b:13, c:12, d:16, e:9, f:5. Merge $f, e \to 14$; $c, b \to 25$; $14, d \to 30$; $25, 30 \to 55$; $a, 55 \to 100$. Codes: $a = 0$, $c = 100$, $b = 101$, $f = 1100$, $e = 1101$, $d = 111$. Cost $= 45 + 3(12 + 13 + 16) + 4(5 + 9) = 224$ (vs. 300 with a fixed 3-bit code).

Lemma 4.7 (Greedy choice)**.** *Let x, y be the two least frequent characters. There is an optimal tree in which x and y are siblings at maximum depth.*

Proof. An optimal tree is full (every internal node has 2 children; otherwise contract). Take siblings a, b at maximum depth. Swap x with a: since $f(x) \le f(a)$ and $\mathrm{depth}(x) \le \mathrm{depth}(a)$, the cost changes by $(f(a) - f(x))(\mathrm{depth}(x) - \mathrm{depth}(a)) \le 0$. Similarly swap y with b. The result is still optimal. □

Lemma 4.8 (Optimal substructure)**.** *Let C' be C with x, y replaced by a new character z, $f(z) = f(x) + f(y)$. If T' is optimal for C', then replacing leaf z in T' by an internal node with children x, y gives an optimal tree T for C.*

Proof. $B(T) = B(T') + f(x) + f(y)$. If some T'' for C had $B(T'') < B(T)$, by the previous lemma we may assume x, y are siblings in T''; merging them gives a tree for C' with cost $B(T'') - f(x) - f(y) < B(T')$, a contradiction. □

Together, by induction on $|C|$, Huffman's algorithm is optimal.

4.5 Pitfalls

- A greedy rule that "works on examples" is not a proof. Always try to find a counterexample first.
- In exchange arguments, make sure the swap keeps the solution *feasible* and argue it does not get worse.

- Coin change with denominations $\{1, 3, 4\}$ and amount 6: greedy gives $4 + 1 + 1$, optimal is $3 + 3$.

Summary

- Interval scheduling: earliest finish time; greedy stays ahead. $\Theta(n \log n)$.
- Minimize max lateness: earliest deadline first; exchange adjacent inversions.
- Fractional knapsack: highest value density first; 0/1 knapsack is *not* greedy.
- Huffman: repeatedly merge the two least frequent; greedy choice + optimal substructure.

Chapter 5

Dynamic Programming

Paradigm

Dynamic programming (DP) solves a problem by combining solutions to **overlapping subproblems**, each solved once and stored in a table. It applies when the problem has **optimal substructure**: an optimal solution is built from optimal solutions to subproblems.

Recipe.

1. Define the subproblems precisely ("$M[i]$ = the optimal value for ...").
2. Write a recurrence for $M[i]$ in terms of smaller subproblems, with base cases. Justify it: consider what an optimal solution does with the last element/choice.
3. Choose an evaluation order so that dependencies are computed first (bottom-up), or memoize (top-down).
4. Runtime = (number of subproblems) $\times$ (time per subproblem). Space is the table size.
5. Recover the actual solution by tracing back through the table (store the choice made, or recompute it).

Example 5.1 (Why tables)**.** Naive recursion for $F(n) = F(n-1) + F(n-2)$ takes $\Theta(\varphi^n)$ because $F(n-k)$ is recomputed many times. Filling $F[0..n]$ left to right takes $\Theta(n)$ additions.

5.1 Weighted interval scheduling

Intervals $i = 1..n$ with weights v_i, sorted by finish time $f_1 \le \cdots \le f_n$. Find a compatible set of max total weight. (Greedy fails once weights are present.) Let $p(j)$ be the largest index $i < j$ with $f_i \le s_j$ (0 if none).

Let $M[j]$ be the max weight using intervals $1..j$. An optimal solution either excludes j, or includes j and then only intervals $1..p(j)$ remain usable:

$$M[0] = 0, \qquad M[j] = \max\big(M[j-1],\ v_j + M[p(j)]\big).$$

```
sort by finish time; compute p(1..n) by binary search    // O(n log n)
M[0] = 0
for j = 1 to n do M[j] = max(M[j-1], v_j + M[p(j)])
// traceback
j = n
while j > 0 do
    if v_j + M[p(j)] >= M[j-1] then output j; j = p(j)
    else j = j - 1
```

Total $\Theta(n \log n)$; the DP itself is $\Theta(n)$.

5.2 0/1 knapsack

Items $i = 1..n$ with integer weights w_i and values v_i; capacity W (integer). Choose S with $\sum_{i \in S} w_i \leq W$ maximizing $\sum_{i \in S} v_i$.

Single index subproblems are not enough: after deciding item n we need to know the *remaining capacity*. Let $K[i][w]$ be the max value using items $1..i$ with capacity w:

$$K[0][w] = 0, \qquad K[i][w] = \begin{cases} K[i-1][w] & w_i > w, \\ \max(K[i-1][w],\ v_i + K[i-1][w-w_i]) & \text{otherwise.} \end{cases}$$

```
for w = 0 to W do K[0][w] = 0
for i = 1 to n do
    for w = 0 to W do
        K[i][w] = K[i-1][w]
        if w_i <= w then K[i][w] = max(K[i][w], v_i + K[i-1][w - w_i])
return K[n][W]
```

Runtime $\Theta(nW)$.

Pseudo-polynomial

$\Theta(nW)$ is *not* polynomial in the input size: W is written with $\log W$ bits, so nW can be exponential in the input length. Such algorithms are called **pseudo-polynomial**. Knapsack (decision version) is NP-complete (Chapter 10), so we do not expect a truly polynomial algorithm.

5.3 Longest common subsequence (LCS)

A subsequence of $X = x_1 \dots x_m$ is obtained by deleting characters (not necessarily contiguous). Given X and $Y = y_1 \dots y_n$, find a longest common subsequence.

Let $C[i][j]$ = length of an LCS of $X[1..i]$ and $Y[1..j]$.

$$C[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0, \\ C[i-1][j-1] + 1 & x_i = y_j, \\ \max(C[i-1][j],\ C[i][j-1]) & x_i \neq y_j. \end{cases}$$

Lemma 5.2. *If $x_i = y_j$, some LCS of $X[1..i], Y[1..j]$ ends by matching x_i with y_j. If $x_i \neq y_j$, an LCS does not use x_i or does not use y_j.*

Proof. Suppose $x_i = y_j$ and Z is an LCS. If Z uses neither x_i nor y_j we can append x_i, contradicting maximality. If it uses only one of them, say x_i is matched with $y_{j'}$ for $j' < j$, re-match x_i with y_j instead (still a valid common subsequence of the same length). The second statement is immediate since x_i and y_j cannot both be the last matched pair. □

Fill row by row; $\Theta(mn)$ time and space. Traceback from $C[m][n]$ recovers the LCS. Only the length needs $\Theta(\min(m, n))$ space (keep two rows); Hirschberg's algorithm recovers the LCS in linear space too.

Example 5.3. $X =$ `ABCBDAB`, $Y =$ `BDCABA`: LCS length 4, e.g. `BCBA`.

5.4 Edit distance

The edit distance between X and Y is the minimum number of insertions, deletions, and substitutions turning X into Y. Let $D[i][j]$ = edit distance between $X[1..i]$ and $Y[1..j]$. Consider the last column of an optimal alignment:

$$D[i][0] = i, \quad D[0][j] = j, \quad D[i][j] = \min \begin{cases} D[i-1][j] + 1 & \text{delete } x_i, \\ D[i][j-1] + 1 & \text{insert } y_j, \\ D[i-1][j-1] + [x_i \neq y_j] & \text{match/substitute.} \end{cases}$$

$\Theta(mn)$ time. (General costs, e.g. gap penalties in sequence alignment, work the same way.)

Example 5.4. `kitten` → `sitting` has distance 3: substitute k/s, substitute e/i, insert g.

5.5 Longest increasing subsequence (LIS)

Given $A[1..n]$, find the longest strictly increasing subsequence. Let $L[i]$ be the length of the longest increasing subsequence *ending at* i:

$$L[i] = 1 + \max\{L[j] : j < i,\ A[j] < A[i]\} \quad (\max \emptyset = 0).$$

The answer is $\max_i L[i]$. Time $\Theta(n^2)$.

$\mathcal{O}(n \log n)$ improvement. Maintain $T[k]$ = the smallest possible last element of an increasing subsequence of length k seen so far. T is strictly increasing, so for each $A[i]$ binary search for the first $T[k] \geq A[i]$ and set $T[k] = A[i]$ (or append if none). The final length of T is the LIS length.

(Alternatively: LIS of A = LCS of A and sort(A) with duplicates removed.)

5.6 Matrix-chain multiplication

Multiplying a $p \times q$ matrix by a $q \times r$ matrix costs pqr scalar multiplications. Given $A_1 \cdots A_n$ where A_i is $d_{i-1} \times d_i$, choose a parenthesization minimizing total cost. (Example: dimensions 10×100, 100×5, 5×50; $(A_1A_2)A_3$ costs $5000 + 2500 = 7500$ while $A_1(A_2A_3)$ costs $25000 + 50000 = 75000$.) The number of parenthesizations is the Catalan number $\Omega(4^n/n^{3/2})$, so brute force is hopeless.

Let $m[i][j]$ = min cost to compute $A_i \cdots A_j$. The last multiplication splits at some k:

$$m[i][i] = 0, \qquad m[i][j] = \min_{i \leq k < j} \big(m[i][k] + m[k+1][j] + d_{i-1}d_k d_j\big).$$

```
for i = 1 to n do m[i][i] = 0
for len = 2 to n do                    // increasing interval length
    for i = 1 to n - len + 1 do
        j = i + len - 1; m[i][j] = ∞
        for k = i to j - 1 do
            c = m[i][k] + m[k+1][j] + d_{i-1} d_k d_j
            if c < m[i][j] then m[i][j] = c; s[i][j] = k
```

$\Theta(n^2)$ subproblems, $\mathcal{O}(n)$ each: $\Theta(n^3)$ time, $\Theta(n^2)$ space. This is the template for **interval DP**: subproblems are contiguous ranges, evaluated by increasing length.

5.7 Optimal binary search trees

Keys $k_1 < \cdots < k_n$ with search probabilities $p_1, \ldots, p_n$. Build a BST minimizing expected search cost $\sum_i p_i(\text{depth}(k_i) + 1)$. (A balanced tree is not optimal when probabilities are skewed; greedy "most likely key at the root" also fails.)

Let $e[i][j]$ be the minimum cost for keys $k_i..k_j$, and $w(i,j) = \sum_{t=i}^{j} p_t$. If k_r is the root, every key in the subtree moves one level deeper, adding $w(i,j)$:

$$e[i][i-1] = 0, \qquad e[i][j] = \min_{i \le r \le j} \big(e[i][r-1] + e[r+1][j]\big) + w(i,j).$$

Same structure as matrix-chain: $\Theta(n^3)$ (Knuth's observation that the optimal root is monotone reduces this to $\Theta(n^2)$). CLRS's version also includes probabilities q_i for unsuccessful searches (dummy leaves); the recurrence is the same with w adjusted.

5.8 Top-down memoization

Instead of ordering the loops, write the recursion directly and cache results:

```
memo = empty table
LCS(i, j):
    if i == 0 or j == 0 then return 0
    if memo[i][j] defined then return memo[i][j]
    if x_i = y_j then r = LCS(i-1, j-1) + 1
    else r = max(LCS(i-1, j), LCS(i, j-1))
    memo[i][j] = r; return r
```

Same asymptotics; only reachable subproblems are computed, at the cost of recursion overhead.

5.9 Pitfalls

- State the meaning of every table entry in words; most wrong DPs come from vague subproblem definitions.
- "Ending at i" vs. "using a prefix up to i" are different subproblems (LIS uses the former, knapsack the latter).
- Check that the evaluation order respects dependencies (interval DP: by length, not by i).
- Count work per subproblem: matrix-chain is $\Theta(n^3)$, not $\Theta(n^2)$.

Summary

Problem	Subproblem	Time
Weighted interval scheduling	best on intervals $1..j$	$\Theta(n \log n)$
0/1 knapsack	items $1..i$, capacity w	$\Theta(nW)$ (pseudo-poly)
LCS / edit distance	prefixes $X[1..i], Y[1..j]$	$\Theta(mn)$
LIS	ending at i	$\Theta(n^2)$ or $\Theta(n \log n)$
Matrix chain / optimal BST	interval $[i, j]$	$\Theta(n^3)$

Chapter 6

Graph Basics, BFS and DFS

6.1 Definitions and representations

A graph $G = (V, E)$ has $n = |V|$ vertices and $m = |E|$ edges. In an undirected graph, $m \le \binom{n}{2}$; in a directed graph, $m \le n(n-1)$. A graph is *connected* if every pair of vertices is joined by a path; a *tree* is a connected acyclic graph (with $n - 1$ edges).

	Adjacency matrix	Adjacency lists
Space	$\Theta(n^2)$	$\Theta(n+m)$
Is $(u, v) \in E$?	$\Theta(1)$	$\Theta(\deg u)$
List neighbours of u	$\Theta(n)$	$\Theta(\deg u)$

We use adjacency lists unless stated; "linear time" for graphs means $\mathcal{O}(n+m)$. Handshake lemma: $\sum_v \deg(v) = 2m$.

6.2 Breadth-first search

BFS explores vertices in order of distance (number of edges) from a source s.

```
BFS(G, s):
    for each v in V do dist[v] = ∞; parent[v] = nil
    dist[s] = 0; Q = queue containing s
    while Q not empty do
        u = Dequeue(Q)
        for each v adjacent to u do
            if dist[v] == ∞ then
                dist[v] = dist[u] + 1; parent[v] = u; Enqueue(Q, v)
```

Each vertex is enqueued at most once and each adjacency list scanned once: $\mathcal{O}(n+m)$.

Theorem 6.1. *After BFS,* $\text{dist}[v] = \delta(s, v)$*, the length of a shortest* s*-*v *path, and following* `parent` *pointers gives a shortest path.*

Proof sketch. Invariant: the queue holds vertices with nondecreasing `dist`, and the first and last differ by at most 1. So vertices are dequeued in nondecreasing order of `dist`. Clearly $\text{dist}[v] \ge \delta(s, v)$ since it is the length of an actual path. Suppose v is a vertex with minimum $\delta(s, v)$ such that $\text{dist}[v] > \delta(s, v)$, and let u be the vertex before v on a shortest path; then $\text{dist}[u] = \delta(s, u) =$

$\delta(s,v)-1$. When u was dequeued, v was either undiscovered (then $\text{dist}[v]=\text{dist}[u]+1$) or already discovered from a vertex dequeued no later than u, giving $\text{dist}[v]\le\text{dist}[u]+1$. Contradiction. □

The parent edges form the **BFS tree**. In an undirected graph every non-tree edge joins vertices whose levels differ by at most 1.

6.2.1 Testing bipartiteness

G is *bipartite* if $V=A\cup B$ with every edge between A and B; equivalently, G is 2-colourable.

Theorem 6.2. *A graph is bipartite if and only if it has no odd cycle.*

Algorithm: run BFS from each unvisited vertex; colour by parity of level. If some edge joins two vertices of the same level, report "not bipartite", otherwise the colouring is a valid bipartition. $\mathcal{O}(n+m)$.

Correctness. If no edge joins same-level vertices, all edges join adjacent levels (BFS property), which have different parities: valid 2-colouring. If edge uv joins two vertices at the same level k, let w be their lowest common ancestor in the BFS tree at level $j<k$. The tree paths $w\to u$ and $w\to v$ plus edge uv form a cycle of length $2(k-j)+1$, odd; so G is not bipartite (an odd cycle cannot be 2-coloured). □

6.3 Depth-first search

DFS goes as deep as possible before backtracking. We record discovery and finish times.

```
DFS(G):
    for each v do colour[v] = white
    time = 0
    for each v in V do
        if colour[v] == white then Visit(v)

Visit(u):
    time = time + 1; d[u] = time; colour[u] = grey
    for each v adjacent to u do
        if colour[v] == white then parent[v] = u; Visit(v)
    colour[u] = black; time = time + 1; f[u] = time
```

Runtime $\Theta(n+m)$. White = undiscovered, grey = on the recursion stack, black = finished.

Theorem 6.3 (Parenthesis theorem)**.** *For any u,v, the intervals $[d[u],f[u]]$ and $[d[v],f[v]]$ are either disjoint or one contains the other; v is a descendant of u in the DFS forest iff $[d[v],f[v]]\subseteq[d[u],f[u]]$.*

Theorem 6.4 (White-path theorem)**.** *v is a descendant of u iff at time $d[u]$ there is a path from u to v consisting entirely of white vertices.*

6.3.1 Edge classification (directed graphs)

When DFS examines edge (u,v):

Type	Colour of v when explored	Intervals
Tree edge	white	v discovered via this edge
Back edge (to ancestor)	grey	$d[v] < d[u] < f[u] < f[v]$
Forward edge (to non-child descendant)	black, $d[u] < d[v]$	$d[u] < d[v] < f[v] < f[u]$
Cross edge	black, $d[v] < d[u]$	$d[v] < f[v] < d[u] < f[u]$

In an undirected graph, every edge is a tree edge or a back edge.

Lemma 6.5. *A directed graph has a cycle iff DFS finds a back edge.*

Proof. A back edge (u, v) plus the tree path $v \rightsquigarrow u$ is a cycle. Conversely, let C be a cycle and v the first vertex of C discovered; let u be its predecessor on C. At $d[v]$ the rest of C is a white path from v to u, so u becomes a descendant of v (white-path theorem), and (u, v) is a back edge. □

6.4 Topological sort

A *topological order* of a DAG lists the vertices so that every edge (u, v) has u before v. A directed graph has one iff it is acyclic.

DFS algorithm: run DFS and output vertices in *decreasing finish time* (prepend each vertex to a list when it finishes).

Correctness. For any edge (u, v) in a DAG we show $f[v] < f[u]$. When (u, v) is explored, v is not grey (that would be a back edge, i.e. a cycle). If v is white, it becomes a descendant of u and finishes first. If v is black, it has already finished, while u has not. □

Kahn's algorithm (alternative): repeatedly remove a vertex of in-degree 0, maintaining in-degree counts and a queue. Also $\mathcal{O}(n + m)$, and it detects a cycle when vertices remain but none has in-degree 0.

6.5 Strongly connected components

In a directed graph, u and v are *strongly connected* if each is reachable from the other. This is an equivalence relation; its classes are the **strongly connected components** (SCCs). Contracting each SCC to a vertex gives the **component graph** G^{SCC}, which is a DAG.

6.5.1 Kosaraju's algorithm

```
SCC(G):
    run DFS(G) to compute finish times f[u]
    compute G^T (reverse all edges)
    run DFS(G^T), with the main loop considering vertices in decreasing f[u]
    each tree of the second DFS forest is one SCC
```

Runtime $\Theta(n + m)$ (two DFS's plus the transpose).

Lemma 6.6. *Let C, C' be distinct SCCs with an edge from C to C'. Then* $\max_{u \in C} f[u] > \max_{v \in C'} f[v]$.

Proof. If the first DFS discovers a vertex x of C first, all of $C \cup C'$ is white-reachable from x, so everything in C' finishes before x. If it discovers C' first, C is not reachable from C' (the component graph is acyclic), so all of C' finishes before any vertex of C is discovered. □

Why it works: the second DFS starts at the vertex with the largest finish time, which lies in a *source* SCC C of G^{SCC}, i.e. a sink of $(G^T)^{SCC}$, so the DFS from it reaches exactly C. In general, by the lemma every edge of G^T leaving the current component goes to a component with a larger maximum finish time, which the second pass has already fully explored (those vertices are no longer white). So each DFS tree in the second pass is exactly one SCC.

6.5.2 Tarjan's algorithm (one pass)

Tarjan's algorithm runs a single DFS, keeping a stack of vertices and a value low$[u]$ = the smallest discovery time reachable from u using tree edges down and then at most one back/cross edge to a vertex still on the stack.

```
Visit(u):
    d[u] = low[u] = ++time; push u on S; onStack[u] = true
    for each v adjacent to u do
        if d[v] undefined then Visit(v); low[u] = min(low[u], low[v])
        elif onStack[v] then low[u] = min(low[u], d[v])
    if low[u] == d[u] then           // u is the root of an SCC
        repeat pop w from S; onStack[w] = false; add w to current SCC
        until w == u
```

Also $\Theta(n+m)$. Tarjan outputs SCCs in reverse topological order of the component graph.

Summary

- BFS: shortest paths in unweighted graphs, bipartiteness (odd cycle iff same-level edge), $\mathcal{O}(n+m)$.
- DFS: discovery/finish times, parenthesis and white-path theorems; tree/back/forward/cross edges.
- Cycle iff back edge; topological order is decreasing finish time.
- SCCs: Kosaraju (DFS, transpose, DFS by decreasing finish time) or Tarjan (low-links), both linear.

Chapter 7

Minimum Spanning Trees

Definition 7.1. Let $G = (V, E)$ be a connected undirected graph with edge weights $w : E \to \mathbb{R}$. A *spanning tree* is a subset $T \subseteq E$ forming a tree on all of V ($|T| = n - 1$). A *minimum spanning tree* (MST) minimizes $w(T) = \sum_{e \in T} w(e)$.

Applications: network design (cheapest way to connect sites), clustering (delete the $k - 1$ heaviest MST edges to get a max-spacing k-clustering), and approximation algorithms (metric TSP, Chapter 11).

7.1 The cut property

A *cut* is a partition $(S, V \setminus S)$ with $\emptyset \neq S \neq V$. An edge *crosses* the cut if it has one endpoint on each side. A set A of edges *respects* the cut if no edge of A crosses it.

Theorem 7.2 (Cut property)**.** *Let A be a subset of some MST, $(S, V \setminus S)$ a cut respecting A, and e a minimum-weight edge crossing the cut. Then $A \cup \{e\}$ is contained in some MST. (In particular, if edge weights are distinct, the lightest edge crossing any cut is in* every *MST.)*

Proof (exchange). Let T be an MST containing A. If $e = uv \in T$ we are done. Otherwise $T \cup \{e\}$ has a unique cycle, which crosses the cut at e and so must cross it again at some edge $e' \in T$, $e' \notin A$ (since A respects the cut). Then $T' = T \cup \{e\} \setminus \{e'\}$ is a spanning tree (removing an edge of the cycle keeps it connected) with $w(T') = w(T) + w(e) - w(e') \leq w(T)$ since $w(e) \leq w(e')$. So T' is an MST containing $A \cup \{e\}$. □

Theorem 7.3 (Cycle property)**.** *If e is a strictly heaviest edge on some cycle, then e is in no MST.*

Generic MST algorithm

Start with $A = \emptyset$. While $|A| < n - 1$, find a cut respecting A and add a lightest crossing edge. By the cut property, A is always contained in an MST. Kruskal and Prim are two ways of choosing the cut.

7.2 Kruskal's algorithm

```
Kruskal(G, w):
    A = {}
```

```
    for each v in V do MakeSet(v)
    sort E by weight, increasing
    for each edge (u, v) in sorted order do
        if Find(u) ≠ Find(v) then          // different components
            A = A ∪ {(u, v)}; Union(u, v)
    return A
```

Correctness: when (u, v) is added, let S be the component of u in (V, A). The cut $(S, V \setminus S)$ respects A, and (u, v) is the lightest crossing edge: any lighter crossing edge would have been considered earlier and added (its endpoints were in different components then, and still are). Edges that are skipped would close a cycle, so the result is a forest, and since G is connected it ends with $n - 1$ edges.

7.2.1 Union-find (disjoint sets)

Operations: `MakeSet(x)`, `Find(x)` (return the representative of x's set), `Union(x,y)`.

- **Linked lists with union by size**: each element stores a pointer to its representative, so `Find` is $\mathcal{O}(1)$; `Union` relabels the smaller set. An element's set at least doubles every time it is relabeled, so each element is relabeled $\leq \log_2 n$ times: total union cost $\mathcal{O}(n \log n)$.
- **Forest with union by rank and path compression**: m operations take $\mathcal{O}(m\,\alpha(n))$, where α is the inverse Ackermann function (≤ 4 in practice).

```
Find(x):
    if parent[x] ≠ x then parent[x] = Find(parent[x])    // path compression
    return parent[x]
Union(x, y):
    a = Find(x); b = Find(y)
    if rank[a] < rank[b] then swap a, b
    parent[b] = a
    if rank[a] == rank[b] then rank[a] = rank[a] + 1
```

Kruskal runtime: sorting $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$ (since $m \leq n^2$), plus $2m$ finds and $n - 1$ unions, which is $\mathcal{O}(m \log n)$ even with the simple list structure. Total $\mathcal{O}(m \log n)$.

7.3 Prim's algorithm

Prim grows a single tree from a root r, always adding the lightest edge leaving the tree (cut $=$ (tree, rest)).

```
Prim(G, w, r):
    for each v do key[v] = ∞; parent[v] = nil
    key[r] = 0
    Q = min-priority queue of all vertices keyed by key
    while Q not empty do
        u = ExtractMin(Q)
        for each v adjacent to u do
            if v in Q and w(u, v) < key[v] then
                parent[v] = u; DecreaseKey(Q, v, w(u, v))
    return {(v, parent[v]) : v ≠ r}
```

key[v] is the weight of the lightest edge from v to the current tree. Correctness is directly the cut property with S = the extracted vertices.

Runtime: n `ExtractMin` and up to m `DecreaseKey`.

Priority queue	Total time	Best for
Unsorted array	$\mathcal{O}(n^2)$	dense graphs
Binary heap	$\mathcal{O}((n+m)\log n) = \mathcal{O}(m \log n)$	sparse graphs
Fibonacci heap	$\mathcal{O}(m + n \log n)$	theory

Example 7.4. Vertices a, b, c, d with edges ab:1, bc:2, ac:3, cd:4, bd:5. Kruskal: take ab (1), bc (2), skip ac (cycle), take cd (4): weight 7. Prim from a: add ab, then lightest edge leaving $\{a, b\}$ is bc, then cd. Same tree.

7.4 Uniqueness and other facts

- If all weights are distinct, the MST is unique.
- Adding a constant to all weights, or applying any increasing function, does not change which trees are MSTs (only the order of edges matters). This is *not* true for shortest paths.
- Negative weights are fine for MST algorithms.
- Borůvka's algorithm: in each round every component adds its lightest outgoing edge; $\mathcal{O}(\log n)$ rounds, $\mathcal{O}(m \log n)$ total.

Summary

- Cut property: the lightest edge across a cut respecting A can be added safely; proved by exchanging with a cycle edge.
- Kruskal: sorted edges + union-find, $\mathcal{O}(m \log n)$.
- Prim: grow one tree using a priority queue, $\mathcal{O}(m \log n)$ with a binary heap, $\mathcal{O}(n^2)$ with an array.

Chapter 8

Shortest Paths

Let $G = (V, E)$ be a directed graph with edge weights w. The weight of a path is the sum of its edge weights, and $\delta(u, v)$ is the minimum weight of a u-v path (∞ if none; $-\infty$ if a negative-weight cycle is reachable on the way).

Problem	Weights	Algorithm	Time
Single source	unweighted	BFS	$\mathcal{O}(n + m)$
Single source	DAG, any weights	topological order	$\mathcal{O}(n + m)$
Single source	$w \geq 0$	Dijkstra	$\mathcal{O}((n + m) \log n)$
Single source	any (detects neg. cycles)	Bellman–Ford	$\mathcal{O}(nm)$
All pairs	any, no neg. cycles	Floyd–Warshall	$\Theta(n^3)$

8.1 Common structure

Lemma 8.1 (Optimal substructure)**.** *Any subpath of a shortest path is a shortest path.*

Proof. If a subpath $x \rightsquigarrow y$ could be replaced by a lighter one, the whole path would get lighter. □

If there is no negative cycle, shortest paths can be taken to be simple (at most $n - 1$ edges): removing a cycle of nonnegative weight does not increase weight.

All single-source algorithms maintain estimates $d[v] \geq \delta(s, v)$ and improve them by **relaxation**:

```
Relax(u, v):
    if d[u] + w(u, v) < d[v] then
        d[v] = d[u] + w(u, v); parent[v] = u
```

Initialize $d[s] = 0$, $d[v] = \infty$ otherwise. Key facts: $d[v]$ never drops below $\delta(s, v)$ (it is always the weight of some path), and (*path-relaxation property*) if $s = v_0, v_1, \ldots, v_k$ is a shortest path and its edges are relaxed in this order (possibly interleaved with other relaxations), then afterwards $d[v_k] = \delta(s, v_k)$.

8.2 Shortest paths in a DAG

```
DAGShortestPaths(G, w, s):
    topologically sort G
    initialize d
```

```
    for each u in topological order do
        for each edge (u, v) do Relax(u, v)
```

Every path visits vertices in topological order, so its edges are relaxed in path order; by path relaxation the result is correct. $\mathcal{O}(n+m)$. Negative weights are fine. Negating weights gives *longest* paths in a DAG (critical path in project scheduling); longest simple path in a general graph is NP-hard.

8.3 Dijkstra's algorithm

For nonnegative weights.

```
Dijkstra(G, w, s):
    initialize d; S = {}
    Q = min-priority queue on V keyed by d
    while Q not empty do
        u = ExtractMin(Q); S = S ∪ {u}
        for each edge (u, v) do Relax(u, v)     // DecreaseKey if d[v] changes
```

Theorem 8.2. *If all weights are nonnegative, then when u is extracted, $d[u] = \delta(s,u)$.*

Proof. Suppose not, and let u be the first vertex extracted with $d[u] > \delta(s,u)$. Then $u \neq s$ and u is reachable. Take a shortest path P from s to u and let (x,y) be the first edge of P with $x \in S$, $y \notin S$. When x was extracted, $d[x] = \delta(s,x)$ (choice of u) and (x,y) was relaxed, so $d[y] = \delta(s,y)$ (subpaths of shortest paths are shortest). Since weights are nonnegative, $\delta(s,y) \le \delta(s,u)$. So

$$d[y] = \delta(s,y) \le \delta(s,u) < d[u],$$

yet u was extracted before y, so $d[u] \le d[y]$. Contradiction. □

Runtime with a binary heap: n extractions and $\le m$ decrease-keys, $\mathcal{O}((n+m)\log n)$; with a Fibonacci heap $\mathcal{O}(m + n\log n)$; with an array $\mathcal{O}(n^2)$.

Negative edges break Dijkstra

Vertices s, a, b with $w(s,a) = 2$, $w(s,b) = 3$, $w(b,a) = -2$. Dijkstra extracts a with $d[a] = 2$ and never updates it, but $\delta(s,a) = 1$. Adding a constant to every edge does *not* fix this: it penalizes paths with more edges.

8.4 Bellman–Ford

```
BellmanFord(G, w, s):
    initialize d
    repeat n - 1 times:
        for each edge (u, v) in E do Relax(u, v)
    for each edge (u, v) in E do
        if d[u] + w(u, v) < d[v] then return "negative cycle reachable from s"
    return d
```

Runtime $\Theta(nm)$.

Theorem 8.3. *If no negative cycle is reachable from s, Bellman–Ford returns $d[v] = \delta(s, v)$ for all v; otherwise it reports a negative cycle.*

Proof. No negative cycle: each reachable v has a shortest simple path $v_0 = s, \dots, v_k = v$ with $k \le n-1$. In round i, edge (v_{i-1}, v_i) is relaxed, so by path relaxation $d[v] = \delta(s, v)$ after round k. Then no edge can be relaxed further ($d[v] \le d[u] + w(u, v)$ is the triangle inequality for δ), so no cycle is reported.

Negative cycle $v_0, \dots, v_k = v_0$ reachable, and suppose no edge is reported: $d[v_i] \le d[v_{i-1}] + w(v_{i-1}, v_i)$ for all i. Summing around the cycle, the d terms cancel (all finite since reachable), giving $0 \le \sum w(v_{i-1}, v_i) < 0$. Contradiction. □

DP view. Let $D_i[v]$ = minimum weight of an s-v path with at most i edges. Then $D_i[v] = \min(D_{i-1}[v],\ \min_{(u,v)\in E} D_{i-1}[u] + w(u, v))$, and $\delta(s, v) = D_{n-1}[v]$. Bellman–Ford is this DP computed in place. *Application*: currency arbitrage (weights $-\log(\text{rate})$; an arbitrage is a negative cycle).

8.5 All-pairs shortest paths: Floyd–Warshall

Running Bellman–Ford from every source is $\mathcal{O}(n^2 m)$, which is $\mathcal{O}(n^4)$ for dense graphs; Dijkstra from every source is $\mathcal{O}(nm \log n)$ but needs $w \ge 0$.

Number the vertices $1..n$. Let $D^{(k)}[i][j]$ be the minimum weight of an i-j path whose *intermediate* vertices all lie in $\{1, \dots, k\}$. A shortest such path either avoids vertex k, or passes through k exactly once (no negative cycles), splitting into $i \rightsquigarrow k$ and $k \rightsquigarrow j$ with intermediates in $\{1..k-1\}$:

$$D^{(0)}[i][j] = \begin{cases} 0 & i = j \\ w(i, j) & (i, j) \in E \\ \infty & \text{else} \end{cases}, \qquad D^{(k)}[i][j] = \min(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]).$$

```
FloydWarshall(W):
    D = W
    for k = 1 to n do
        for i = 1 to n do
            for j = 1 to n do
                if D[i][k] + D[k][j] < D[i][j] then
                    D[i][j] = D[i][k] + D[k][j]; next[i][j] = next[i][k]
    return D
```

$\Theta(n^3)$ time, $\Theta(n^2)$ space (updating in place is safe since row k and column k do not change in round k). A negative cycle exists iff some $D[i][i] < 0$ at the end. The same recurrence with Boolean operations computes the **transitive closure** (Warshall).

Johnson's algorithm handles sparse graphs with negative edges: add a new vertex q with 0-weight edges to all vertices, run Bellman–Ford from q to get $h(v) = \delta(q, v)$, reweight $w'(u, v) = w(u, v) + h(u) - h(v) \ge 0$ (path weights change by $h(\text{start}) - h(\text{end})$, so shortest paths are preserved), then run Dijkstra from every vertex. Total $\mathcal{O}(nm \log n)$.

Summary

- All algorithms are built on relaxation; correctness via the path-relaxation property or an exchange/contradiction argument.
- DAG: relax in topological order, linear time, any weights.
- Dijkstra: greedy, needs nonnegative weights, $\mathcal{O}(m \log n)$.
- Bellman–Ford: $n-1$ rounds of relaxing all edges, $\mathcal{O}(nm)$; extra round detects negative cycles.
- Floyd–Warshall: DP over allowed intermediate vertices, $\Theta(n^3)$.

Chapter 9

Polynomial-Time Reductions and NP

So far we found efficient algorithms. For many natural problems (e.g. 0/1 knapsack without a pseudo-polynomial bound, travelling salesman, 3-colouring) nobody knows a polynomial-time algorithm. The theory of NP-completeness lets us show that such problems are all equally hard, so that a fast algorithm for one would give fast algorithms for all.

9.1 Decision problems and P

Definition 9.1. A *decision problem* is a problem whose answer is yes or no. Formally it is a set L of (encoded) instances, the yes-instances; an algorithm *solves* L if it outputs yes exactly on inputs in L.

Optimization problems have decision versions obtained by adding a threshold:

- Independent-Set: given G and k, does G have an independent set of size $\geq k$?
- TSP: given complete weighted graph and B, is there a tour of total weight $\leq B$?

The decision version is no harder than the optimization version, and usually not much easier: e.g. binary search on k finds the optimal value with $\mathcal{O}(\log n)$ calls, and self-reducibility recovers an optimal solution.

Definition 9.2. **P** is the class of decision problems solvable in time $\mathcal{O}(n^c)$ for some constant c, where n is the input size in bits.

Why polynomial? The class is closed under composition: an algorithm that runs in polynomial time and calls a polynomial-time subroutine polynomially many times is still polynomial. It is also robust across reasonable machine models, and in practice polynomial algorithms usually have small exponents.

Input size, again

Knapsack with $\Theta(nW)$ is not known to be in P: W takes $\log W$ bits. Similarly, trial division to test primality of N takes $\sqrt{N} = 2^{(\log N)/2}$ steps, which is exponential. (Primality *is* in P, by AKS 2002.)

9.2 Polynomial-time reductions

Definition 9.3 (Many-one / Karp reduction)**.** $A \leq_P B$ if there is a polynomial-time computable function f mapping instances of A to instances of B such that

$$x \in A \iff f(x) \in B.$$

Lemma 9.4. *1. If $A \leq_P B$ and $B \in \mathbf{P}$, then $A \in \mathbf{P}$.*

2. (Contrapositive) If $A \leq_P B$ and $A \notin \mathbf{P}$, then $B \notin \mathbf{P}$.

3. $\leq_P$ is transitive.

Proof. (1) On input x, compute $f(x)$ in time $p(|x|)$; then $|f(x)| \leq p(|x|)$, and run B's algorithm in time $q(p(|x|))$, a polynomial. (3) Compose the two functions; a polynomial of a polynomial is a polynomial. □

Direction

$A \leq_P B$ means "A is no harder than B". To show a new problem X is hard, reduce a *known hard* problem *to* X. A reduction must handle *every* instance of A, and you must prove *both* directions of the iff.

(Sometimes Cook/Turing reductions are used: solve A in polynomial time given an oracle for B. For NP-completeness, CS 341 uses Karp reductions.)

9.2.1 First examples

Definition 9.5. In a graph $G = (V, E)$: $S \subseteq V$ is *independent* if no edge has both ends in S; $C \subseteq V$ is a *vertex cover* if every edge has at least one end in C; $K \subseteq V$ is a *clique* if every pair in K is adjacent.

Lemma 9.6. *S is independent in G iff $V \setminus S$ is a vertex cover iff S is a clique in the complement $\overline{G}$.*

Proof. S independent $\iff$ no edge inside S $\iff$ every edge has an end outside S $\iff$ $V \setminus S$ covers every edge. And no edge of G inside S $\iff$ every pair in S is an edge of $\overline{G}$. □

Theorem 9.7. *Independent-Set $\equiv_P$ Vertex-Cover $\equiv_P$ Clique.*

Proof. $(G, k) \in$ IS $\iff$ $(G, n-k) \in$ VC, and $(G, k) \in$ IS $\iff$ $(\overline{G}, k) \in$ Clique. These maps are computable in polynomial time ($\overline{G}$ takes $\mathcal{O}(n^2)$). The reverse reductions are symmetric. □

Example 9.8 (Reduction to a more general problem)**.** Vertex-Cover$\leq_P$Set-Cover: given (G, k), let the universe be E and for each vertex v create the set of edges incident to v. A set cover of size k is exactly a vertex cover of size k.

Example 9.9 (Hamiltonian cycle to TSP)**.** Ham-Cycle: does G have a cycle through every vertex exactly once? Given G, build the complete graph on V with $w(u, v) = 1$ if $uv \in E$ and 2 otherwise, and set $B = n$. A tour of weight $\leq n$ uses only weight-1 edges, i.e. is a Hamiltonian cycle in G; conversely a Hamiltonian cycle is a tour of weight n.

9.3 NP

Definition 9.10. A decision problem L is in **NP** if there is a polynomial-time algorithm V (the *verifier*) and a polynomial p such that

$$x \in L \iff \exists\, y \text{ with } |y| \le p(|x|) \text{ such that } V(x, y) \text{ accepts.}$$

Such a y is a *certificate* (witness) for x.

In words: yes-instances have short proofs that can be checked quickly. No-instances need not have short proofs. (NP stands for *nondeterministic polynomial time*: a nondeterministic machine can guess y and run V.)

Example 9.11.

- Independent-Set∈NP: certificate is the set S; check $|S| \ge k$ and no edge inside S in $\mathcal{O}(n+m)$.
- Ham-Cycle∈NP: certificate is an ordering of V; check it is a permutation and consecutive vertices (cyclically) are adjacent.
- SAT∈NP: certificate is a truth assignment; evaluate the formula.
- Composite (N is not prime) ∈NP: certificate is a nontrivial factor.

Theorem 9.12. $\mathbf{P} \subseteq \mathbf{NP}$.

Proof. If $L \in \mathbf{P}$ via algorithm A, take the verifier $V(x, y) = A(x)$ that ignores the certificate (use the empty certificate). □

Also $\mathbf{NP} \subseteq \mathbf{EXP}$: try all $2^{p(n)}$ certificates.

co-NP. $\mathbf{coNP} = \{\overline{L} : L \in \mathbf{NP}\}$: problems whose *no*-instances have short certificates, e.g. Tautology, or "G has no Hamiltonian cycle". It is unknown whether $\mathbf{NP} = \mathbf{coNP}$.

The big question

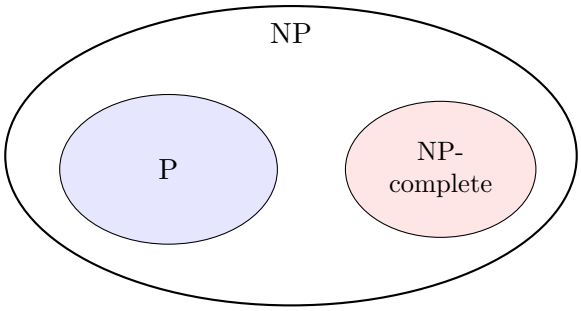
Is $\mathbf{P} = \mathbf{NP}$? I.e. is finding a solution as easy as checking one? It is widely believed that $\mathbf{P} \neq \mathbf{NP}$, but this is open (a Clay Millennium Prize problem).

9.4 NP-hard and NP-complete

Definition 9.13. B is **NP-hard** if $A \le_P B$ for every $A \in \mathbf{NP}$. B is **NP-complete** if moreover $B \in \mathbf{NP}$.

Lemma 9.14. *If some NP-complete problem is in P, then* $\mathbf{P} = \mathbf{NP}$. *If A is NP-complete, $B \in \mathbf{NP}$, and $A \le_P B$, then B is NP-complete.*

Proof. The first follows from the reduction lemma. For the second, any $L \in \mathbf{NP}$ has $L \le_P A \le_P B$, and $\le_P$ is transitive. □



The picture if $\mathbf{P} \neq \mathbf{NP}$ (Ladner: then there are also NP problems that are neither in P nor NP-complete).

Recipe: proving X is NP-complete

1. Show $X \in \mathbf{NP}$: describe the certificate and the polynomial-time verifier.
2. Choose a known NP-complete problem Y.
3. Give a polynomial-time construction f mapping instances of Y to instances of X.
4. Prove: $y \in Y \Rightarrow f(y) \in X$ (turn a solution of y into a solution of $f(y)$).
5. Prove: $f(y) \in X \Rightarrow y \in Y$ (turn a solution of $f(y)$ back into a solution of y). This is usually the harder direction.
6. Argue f runs in polynomial time (size of the construction).

Summary

- Decision problems; P = polynomial time in the number of input bits.
- $A \leq_P B$: polynomial-time map preserving yes/no answers; reduce *from* the known hard problem.
- NP = problems with polynomial-size, polynomial-time verifiable certificates for yes-instances; $\mathbf{P} \subseteq \mathbf{NP}$.
- NP-complete = in NP and every NP problem reduces to it. IS, VC, Clique are polynomially equivalent.

Chapter 10

NP-Completeness

To get started we need one problem proven NP-complete from scratch; after that, everything is done by reductions.

10.1 Satisfiability and the Cook–Levin theorem

A Boolean formula in *conjunctive normal form* (CNF) is an AND of *clauses*, each an OR of *literals* (x_i or $\overline{x_i}$).

- SAT: given a CNF formula Φ, is there a truth assignment making Φ true?
- 3-SAT: the same, where every clause has exactly 3 literals.

Example: $(x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_3} \vee x_2 \vee x_1)$ is satisfied by $x_1 = x_2 = x_3 =$ true.

Theorem 10.1 (Cook 1971, Levin 1973)**.** *SAT is NP-complete.*

Idea only. SAT is in NP (the assignment is the certificate). For hardness, let $L \in \mathbf{NP}$ with verifier V running in time $p(n)$. Given x, build a formula whose variables describe a $p(n) \times p(n)$ *tableau*: the full configuration (tape contents, head position, state) of V at each time step, with the certificate bits left free. Clauses enforce that row 0 is the start configuration on (x, y), each row follows from the previous by the transition rules (a local check on small windows), and some row is accepting. The formula has polynomial size, and it is satisfiable iff some certificate y makes V accept, i.e. iff $x \in L$. □

In CS 341 the proof is not examined; you need the statement and how to use it.

Reduction tree used in this chapter

$$\text{SAT} \le_P \text{3-SAT} \le_P \begin{cases} \textsc{Independent-Set} \le_P \textsc{Vertex-Cover},\ \textsc{Clique} \\ \textsc{Dir-Ham-Cycle} \le_P \textsc{Ham-Cycle} \le_P \text{TSP} \\ \textsc{Subset-Sum} \le_P \textsc{Knapsack} \\ \text{3-}\textsc{Colouring} \end{cases}$$

Every problem listed is in NP (certificates: the assignment / set / cycle / subset / colouring), so all are NP-complete.

10.2 SAT $\leq_P$ 3-SAT

Replace each clause C by an equisatisfiable set of 3-clauses, using fresh variables per clause.

- $|C| = 3$: keep it.
- $|C| = 1$, $C = (\ell)$: with fresh a, b, use $(\ell \vee a \vee b)(\ell \vee a \vee \overline{b})(\ell \vee \overline{a} \vee b)(\ell \vee \overline{a} \vee \overline{b})$; all four hold iff ℓ is true.
- $|C| = 2$, $C = (\ell_1 \vee \ell_2)$: with fresh a, use $(\ell_1 \vee \ell_2 \vee a)(\ell_1 \vee \ell_2 \vee \overline{a})$.
- $|C| = k > 3$, $C = (\ell_1 \vee \cdots \vee \ell_k)$: with fresh $y_1, \ldots, y_{k-3}$, use

$$(\ell_1 \vee \ell_2 \vee y_1)(\overline{y_1} \vee \ell_3 \vee y_2)(\overline{y_2} \vee \ell_4 \vee y_3) \cdots (\overline{y_{k-3}} \vee \ell_{k-1} \vee \ell_k).$$

Correctness of the long-clause gadget. ($\Rightarrow$) If some ℓ_i is true, set y_j = true for $j \leq i-2$ and false for $j \geq i-1$. Each 3-clause before the one containing ℓ_i is satisfied by its y_j, the one containing ℓ_i by ℓ_i, and each after it by its $\overline{y_j}$. ($\Leftarrow$) If all ℓ_i are false, the first clause forces y_1, then the second forces y_2, …, so y_{k-3} is true and the last clause is false. □

The new formula has $\mathcal{O}$(size of Φ) clauses, so the reduction is polynomial.

10.3 3-SAT $\leq_P$ Independent Set

Given Φ with m clauses, build G:

- for each clause, a triangle with one vertex per literal occurrence;
- an edge between every pair of vertices labelled with complementary literals x_i and $\overline{x_i}$.

Set $k = m$.

Proof. ($\Rightarrow$) Given a satisfying assignment, pick one true literal from each clause; the m chosen vertices lie in different triangles and are never complementary (both are true), so they are independent. ($\Leftarrow$) An independent set of size m has exactly one vertex per triangle. It never contains both x_i and $\overline{x_i}$, so set every chosen literal true (other variables arbitrarily); this is consistent and satisfies every clause. □

G has $3m$ vertices and $\mathcal{O}(m^2)$ edges. Hence Independent-Set is NP-complete, and by Chapter 9 so are Vertex-Cover and Clique.

10.4 Hamiltonian cycle

10.4.1 3-SAT $\leq_P$ Directed Hamiltonian Cycle

Let Φ have variables $x_1..x_n$ and clauses $C_1..C_m$.

- For each variable x_i, a bidirected path P_i of $3m+3$ vertices $v_{i,1}, \ldots, v_{i,3m+3}$ (edges both ways between consecutive vertices). Traversing P_i left to right means x_i = true; right to left means false.
- A start s and end t: edges $s \to v_{1,1}$, $s \to v_{1,3m+3}$; from both ends of P_i to both ends of P_{i+1}; from both ends of P_n to t; and $t \to s$.

- For each clause C_j, a vertex c_j. If $x_i \in C_j$, add $v_{i,3j} \to c_j \to v_{i,3j+1}$ (usable when going left to right). If $\overline{x_i} \in C_j$, add $v_{i,3j+1} \to c_j \to v_{i,3j}$ (usable when going right to left).

Proof. ($\Rightarrow$) Traverse each P_i in the direction given by the assignment. For each clause pick one true literal and detour through c_j at that variable's path; the detour goes the right way by construction. This visits everything exactly once.

($\Leftarrow$) Suppose a Hamiltonian cycle enters c_j from $v_{i,3j}$. It must leave to $v_{i,3j+1}$: otherwise $v_{i,3j+1}$ could only be entered from $v_{i,3j+2}$ (and $v_{i,3j}$ is used up), and then it would be a dead end. The spacing (vertices $3j-1$, $3j+2$ are buffers never adjacent to clause vertices) makes this argument work. So removing the clause vertices from the cycle leaves a cycle that runs $s, P_1, P_2, \ldots, P_n, t$, traversing each P_i in one direction. Set x_i by that direction. Each c_j was visited by a detour that is only possible if the corresponding literal is true, so every clause is satisfied. □

10.4.2 Directed $\leq_P$ undirected Hamiltonian cycle

Replace each vertex v by a path $v_{\text{in}} - v_{\text{mid}} - v_{\text{out}}$ and each arc (u, v) by the edge $u_{\text{out}}v_{\text{in}}$. Since v_{mid} has degree 2, any Hamiltonian cycle passes $v_{\text{in}}, v_{\text{mid}}, v_{\text{out}}$ consecutively, so it follows arcs of the directed graph consistently in one direction.

10.4.3 Ham-Cycle $\leq_P$ TSP

Shown in Chapter 9 (weights 1 for edges, 2 for non-edges, bound n). So TSP is NP-complete; in fact this construction satisfies the triangle inequality, so even *metric* TSP is NP-complete. Ham-Path is NP-complete too (similar gadget).

10.5 3-SAT $\leq_P$ Subset Sum

Subset-Sum: given positive integers $a_1..a_r$ and a target T, is there a subset summing to exactly T?

Construct numbers written in base 10 with $n+m$ digits: one digit per variable, then one per clause.

- For each variable x_i: numbers y_i and z_i, both with a 1 in variable digit i. y_i has a 1 in clause digit j for each clause containing x_i; z_i has a 1 in clause digit j for each clause containing $\overline{x_i}$.
- For each clause C_j: slack numbers g_j and h_j with a 1 and a 2 respectively in clause digit j, zero elsewhere.
- Target T: digit 1 in every variable position, digit 4 in every clause position.

Each column sums to at most $5 < 10$, so there are no carries and we can reason digit by digit.

Proof. ($\Rightarrow$) Take y_i if x_i is true, z_i otherwise. Variable digits are 1. Clause digit j gets the number of true literals in C_j, between 1 and 3; add slack (g_j, h_j, or both) to reach 4. ($\Leftarrow$) Variable digit i forces exactly one of y_i, z_i; set x_i accordingly. The slacks contribute at most 3 to clause digit j, so at least one chosen literal number contributes, i.e. some literal of C_j is true. □

The numbers have $n+m$ digits, i.e. $\mathcal{O}(n+m)$ bits each: polynomial. (This is why subset sum's $\mathcal{O}(rT)$ DP does not contradict NP-completeness: T is exponential in the input size.) Corollaries: Knapsack (set $w_i = v_i = a_i$, $W = V = T$) and Partition are NP-complete.

10.6 3-SAT $\leq_P$ 3-Colouring

3-Colouring: can the vertices be coloured with 3 colours so adjacent vertices differ? (2-colouring is in P, by BFS.)

- **Palette**: a triangle on vertices T, F, B (true, false, base). In any 3-colouring they get distinct colours; name colours after them.
- **Variables**: for each x_i a triangle $x_i, \overline{x_i}, B$. So one of $x_i, \overline{x_i}$ is coloured T and the other F.
- **OR-gadget** $\mathrm{OR}(u, v)$: new vertices p, q, o forming a triangle, with edges up and vq. Output o.
- **Clauses**: for $C = (\ell_1 \vee \ell_2 \vee \ell_3)$, build $o_1 = \mathrm{OR}(\ell_1, \ell_2)$ and $o_2 = \mathrm{OR}(o_1, \ell_3)$, and join o_2 to F and B (forcing o_2 to be coloured T).

Lemma 10.2 (OR-gadget)**.** *If u, v are both coloured F, then o must be F. If at least one of u, v is T (the other T, F, or B), the gadget can be coloured with $o = T$.*

Proof. If $u = v = F$, then $p, q \neq F$, so $\{p, q\} = \{T, B\}$ and $o = F$. If $u = T$: put $o = T$; q must avoid v and T, and p must avoid T and q: choose $q \in \{F, B\} \setminus \{v\}$ and p the other of $\{F, B\}$. The case $v = T$ is symmetric. □

Correctness. ($\Rightarrow$) Colour literals by the satisfying assignment. In each clause some literal is T: if $\ell_3 = T$, colour the first gadget validly (always possible: its triangle avoids at most two input colours) and the second with $o_2 = T$; if ℓ_1 or ℓ_2 is T, colour $o_1 = T$ and then $o_2 = T$. ($\Leftarrow$) If all literals in some clause are F, the lemma forces $o_1 = F$ and then $o_2 = F$, contradicting the edge o_2F. So the assignment "x_i true iff coloured T" satisfies every clause. □

The graph has $3 + 3n + 6m$ vertices: polynomial. So 3-Colouring (and k-colouring for every $k \geq 3$) is NP-complete.

10.7 Choosing a source problem

If the target problem is about...	try reducing from
choosing a subset with no conflicts	Independent Set
covering / hitting requirements	Vertex Cover, Set Cover
ordering / sequencing / tours	Ham Cycle or Ham Path
numbers adding up exactly	Subset Sum, Partition
assigning labels with constraints	3-Colouring, 3-SAT
anything else	3-SAT

Common pitfalls: reducing in the wrong direction; proving only one direction of the iff; constructions that are exponential in size (e.g. writing numbers in unary that were given in binary); forgetting to show membership in NP.

Summary

- Cook–Levin: SAT is NP-complete. SAT $\leq_P$ 3-SAT by splitting clauses with fresh variables.
- 3-SAT $\leq_P$ IS (triangles + complement edges, $k = m$); IS $\equiv_P$ VC $\equiv_P$ Clique.
- 3-SAT $\leq_P$ Dir-Ham-Cycle (variable paths, clause detours) $\leq_P$ Ham-Cycle $\leq_P$ TSP.
- 3-SAT $\leq_P$ Subset Sum (base-10 digits, slack numbers 1 and 2, target $1 \ldots 1\, 4 \ldots 4$).
- 3-SAT $\leq_P$ 3-Colouring (palette triangle, variable triangles, OR-gadgets).

Chapter 11

Undecidability and Coping with Hardness

NP-complete problems are believed to need exponential time, but they can at least be solved by brute force. Some problems cannot be solved by *any* algorithm at all.

11.1 Decidability

Definition 11.1. A decision problem is **decidable** if some algorithm (Turing machine, or program in any general-purpose language) halts on every input and outputs the correct yes/no answer. Otherwise it is **undecidable**.

A counting argument already shows undecidable problems exist: there are countably many programs (finite strings) but uncountably many decision problems (subsets of $\{0,1\}^*$). The interesting fact is that natural problems are undecidable.

11.2 The halting problem

HALT: given (the source code of) a program P and an input x, does P halt on x?

Theorem 11.2 (Turing 1936)**.** *HALT is undecidable.*

Proof. Suppose an algorithm $H(P, x)$ always halts and returns true iff P halts on x. Write the program

```
D(P):
    if H(P, P) then loop forever
    else return
```

Run D on its own source. If $D(D)$ halts, then $H(D, D)$ is true, so $D(D)$ loops forever. If $D(D)$ loops forever, then $H(D, D)$ is false, so $D(D)$ returns. Either way we have a contradiction, so H cannot exist. □

This is a *diagonalization* argument, like Cantor's proof that $\mathbb{R}$ is uncountable: D differs from every program P on input P.

11.3 Reductions for undecidability

If A is undecidable and $A \leq B$ via a *computable* (not necessarily polynomial) map f with $x \in A \iff f(x) \in B$, then B is undecidable; otherwise a decider for B composed with f would decide A. Turing reductions (deciding A using a hypothetical decider for B as a subroutine) work too.

Example 11.3 (Halting on empty input)**.** HALT-ε: does P halt on the empty input? Reduce from HALT: given (P, x), build P_x that ignores its input and runs P on x (hard-code x). Then P_x halts on ε iff P halts on x.

Example 11.4 (Program equivalence)**.** EQUIV: do programs P_1, P_2 produce the same output on every input? Reduce from HALT-ε: given P, let $P_1(y) =$ "run P on ε, then output 1" and $P_2(y) =$ "output 1". They are equivalent iff P halts on ε.

Example 11.5 (Does P ever output 0?)**.** Given P, build $Q(y) =$ "run P on ε; output 0". Q ever outputs 0 iff P halts on ε.

Theorem 11.6 (Rice's theorem, statement)**.** *Every nontrivial property of the* function computed *by a program (true for some programs and false for others, depending only on the input/output behaviour) is undecidable.*

Other undecidable problems: Hilbert's tenth problem (does a polynomial equation with integer coefficients have an integer solution?), Post's correspondence problem, whether a context-free grammar is ambiguous, and tiling the plane with a given set of Wang tiles.

Hierarchy

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXP} \subsetneq \text{decidable} \subsetneq \text{all problems.}$$

NP-complete problems are decidable (in exponential time); HALT is not decidable at all. Note that HALT is NP-hard (any decidable problem reduces to it), but it is not in NP.

11.4 Coping with NP-hardness

Showing a problem is NP-complete does not mean we give up. Options:

11.4.1 Special cases

Restricted inputs may be easy: Independent Set on trees or interval graphs (greedy/DP), 2-SAT (via SCCs of the implication graph, linear time), 2-colouring (BFS), Vertex Cover on bipartite graphs (matching), knapsack with small W (pseudo-polynomial DP).

11.4.2 Exact exponential algorithms

Do better than brute force:

- **Backtracking / branch and bound**: explore partial solutions, prune those that cannot lead to a solution or beat the best found so far.
- **DP over subsets**: Held–Karp solves TSP in $\mathcal{O}(n^2 2^n)$ instead of $\mathcal{O}(n!)$, with $D[S][v] =$ shortest path from 1 visiting exactly S and ending at v: $D[S][v] = \min_{u \in S \setminus \{v\}} D[S \setminus \{v\}][u] + w(u, v)$.

- **Fixed-parameter tractability**: Vertex Cover of size k in $\mathcal{O}(2^k m)$: pick any edge uv; one of u, v is in the cover, so branch on both, recursing with $k-1$. Depth k, branching factor 2.

11.4.3 Approximation algorithms

An α-approximation (for minimization) runs in polynomial time and always returns a solution of cost $\leq \alpha \cdot \mathrm{OPT}$.

- **Vertex Cover, factor 2**: repeatedly pick an uncovered edge uv and add both u and v. The picked edges form a matching M; any cover contains an endpoint of each matching edge, so $\mathrm{OPT} \geq |M|$, while we output $2|M|$.
- **Metric TSP, factor 2**: compute an MST T (Chapter 7), walk around it (each edge twice), and shortcut repeated vertices. Removing one edge of an optimal tour gives a spanning tree, so $w(T) \leq \mathrm{OPT}$; the walk costs $2w(T)$ and shortcuts don't increase cost by the triangle inequality. (Christofides improves this to 3/2.)
- **General TSP** has no constant-factor approximation unless **P** = **NP**: with weights 1 for edges and $\alpha n + 1$ for non-edges, an α-approximation would decide Ham-Cycle.
- **Knapsack** has an FPTAS: scale and round values, then run the value-indexed DP, giving $(1-\varepsilon)$-optimal solutions in time polynomial in n and $1/\varepsilon$.

11.4.4 Heuristics

Local search, simulated annealing, SAT solvers, integer programming solvers: no worst-case guarantees but often excellent in practice.

Summary

- Halt is undecidable by diagonalization; reductions (computable maps) spread undecidability.
- Undecidable is strictly worse than NP-complete: NP-complete problems are solvable in exponential time.
- Coping with NP-hardness: special cases, smarter exponential algorithms (Held–Karp, FPT), approximation (2-approx VC and metric TSP), heuristics.